

Car Booking Application Remediation Report

Date prepared: 25 September 2025

Team members: _____ (update with final submission names).

1. Introduction

This report documents the security hardening work completed on the PHP-based car booking application originally obtained from a public GitHub repository. The assessment targeted common web weaknesses across authentication, booking, and administration workflows, then implemented layered mitigations aligned with OWASP and SE4030 course expectations. The objectives were to identify at least seven distinct vulnerabilities, remediate them without regressing core functionality, introduce Google OAuth 2.0 sign-in, and provide operational guidance for the improved release.

2. Methodology

The team combined manual secure code review, functional smoke testing in a local LAMP stack, and targeted static analysis to isolate exploitable patterns. Vulnerabilities were validated by reproducing attacks against the unpatched baseline, then re-tested after mitigation using prepared statements, CSRF token verification, and session observation through browser developer tools. Syntax safety of the updated PHP code was confirmed with the built-in linter ('php -l') prior to release packaging.

3. Remediated Vulnerabilities Overview

Table 1 summarises the seven high-priority issues addressed during the sprint. Detailed evidence and remediation notes follow in Section 4.

- V1 â SQL injection in user login allowed credential bypass through crafted email payloads (High severity) â fixed by enforcing parameterised queries in 'login.php'.
- V2 â Registration endpoint SQL injection and user enumeration (High) â mitigated via prepared statements, strict validation, and contextual messaging in 'register.php'.
- V3 â Booking workflow SQL injection and transaction tampering (High) â addressed by validating IDs, constraining dates, sanitising comments, and inserting with bound parameters in 'carDetails.php'.
- V4 â Stored and reflected cross-site scripting across vehicle listings and bookings (High) â resolved by introducing a reusable HTML escaping helper and sanitising free-form input across templates.
- V5 â Absence of CSRF protection on authentication and booking forms (High) â remediated with per-session anti-CSRF tokens issued by 'inc/security.php'.
- V6 â Weak password hashing and missing upgrade path (High) â strengthened by migrating to

'password_hash'/'password_verify' with automatic MD5 rehashing for legacy rows.

- V7 â Session fixation and insecure cookie attributes (High) â mitigated via 'session_regenerate_id', HTTP-only cookies, and SameSite restrictions in the shared security bootstrap.

4. Detailed Vulnerability Analysis and Fixes

V1. SQL Injection in User Login

Original issue: The previous login implementation concatenated the submitted email address directly into the SQL statement, allowing attackers to authenticate without knowing passwords by injecting logical operators (e.g., '\ OR 1=1 --).

Impact: Full account compromise for any registered user, enabling data exfiltration and fraudulent bookings.

Remediation: 'login.php' now prepares the lookup query with 'bind_param', validates that the identifier is a well-formed email address, and blocks login for OAuth-managed accounts. Password verification also upgrades legacy MD5 hashes to modern 'password_hash' storage once a user successfully authenticates.

Verification: Attempting the prior payload now returns the generic login failure alert, and the database logs show parameterised execution without injected operators. The PHP syntax check ('php -l login.php') passes without errors.

V2. Registration SQL Injection and Enumeration

Original issue: The registration form interpolated names, emails, and phone numbers directly into an INSERT statement, enabling SQL injection to create arbitrary accounts or corrupt schema state. The flow also leaked whether an email already existed, which facilitated account enumeration for credential stuffing.

Impact: Attackers could mass-register privileged roles, tamper with other tables, or enumerate legitimate users for targeted phishing.

Remediation: 'register.php' now filters and normalises inputs, enforces password complexity, and executes both uniqueness checks and the final insert using prepared statements. If an address already belongs to an OAuth user, the response instructs the visitor to authenticate via the linked provider, removing the oracle.

Verification: SQL map-style payloads are treated as plain text and stored safely, while enumeration attempts receive a single generic error toast. Linting via 'php -l register.php' confirms the script integrity.

V3. Booking Workflow Manipulation

Original issue: The booking submission accepted arbitrary vehicle identifiers, dates, and messages, interpolating them directly into SQL without validation. Attackers could book nonexistent vehicles, tamper with another customer's reservation, or inject SQL/JavaScript

through the free-form comment field.

Impact: Data corruption of the 'carbooking' table, denial of service through invalid reservations, and stored XSS against administrators reviewing bookings.

Remediation: 'carDetails.php' now validates the 'id' query parameter, confirms the vehicle exists, checks chronological date order, caps message length, strips markup, and persists the request via a prepared statement inside a try/catch block that logs operational failures.

Verification: Invalid IDs redirect back to the catalogue, reversed dates are rejected client-side, and automated SQL injection strings are neutralised. Syntax verification succeeded with 'php -l carDetails.php' during testing.

V4. Cross-Site Scripting in Listings and Account Views

Original issue: Dynamic values such as vehicle titles, pricing, and booking metadata were echoed directly into HTML attributes and text nodes. User-controlled data (e.g., booking comments) therefore executed as JavaScript in any visitor's browser.

Impact: Stored or reflected XSS enabled full session hijacking, credential theft, and phishing against customers and administrators.

Remediation: A shared 'escape()' helper in 'inc/security.php' centralises HTML encoding, and all templates now wrap dynamic output with this function. Booking comments are stripped to plain text before storage, eliminating script injection at the source.

Verification: Previously exploitable payloads now render as inert text across 'cars.php', 'carDetails.php', and 'my_account.php', and browser dev tools confirm the DOM no longer contains untrusted <script> nodes.

V5. Missing CSRF Protection on Critical Forms

Original issue: The login, registration, admin login, and booking forms trusted the browser's cookie implicitly. Attackers could trick authenticated users into submitting hidden forms, forcing unintended logins or reservations.

Impact: Cross-site request forgery could lock victims out of accounts, create fraudulent bookings, or escalate privileges in the admin panel.

Remediation: 'inc/security.php' issues high-entropy CSRF tokens per session. Each sensitive form embeds the token and validates it server-side before processing, aborting the request if validation fails.

Verification: Intercepting requests with the wrong or missing token now yields SweetAlert or JavaScript alerts explaining the invalid session, and the booking is not committed.

V6. Weak Password Hashing and Legacy Storage

Original issue: Both user and admin credentials were stored with unsalted MD5 hashes, which

are trivially reversible with rainbow tables. No migration plan was in place to improve existing rows without breaking logins.

Impact: Database compromise would immediately expose plaintext-equivalent passwords for reuse across other services.

Remediation: Authentication handlers now rely on 'password_hash'/'password_verify'. When a legitimate user signs in with an old MD5 hash, the system transparently upgrades the record to a modern algorithm. This logic covers both 'users' and 'admin' tables so no credential remains on MD5 after first use.

Verification: Fresh registrations generate bcrypt hashes, and legacy accounts transition automatically as demonstrated in phpMyAdmin during testing.

V7. Session Fixation and Cookie Hardening

Original issue: Sessions were reused across authentication boundaries, and cookies were missing 'HttpOnly', 'SameSite', and conditional 'Secure' attributes. Attackers could pre-seed a victim with a known session ID or steal cookies through XSS to hijack accounts.

Impact: Compromised sessions undermine every other authentication control, enabling full account takeover.

Remediation: The new 'inc/security.php' bootstrap enforces strict cookie parameters before calling 'session_start()' and regenerates the session identifier after successful logins (including OAuth). This neutralises fixation and prevents JavaScript from reading session tokens.

Verification: Browser developer tools confirm the regenerated session ID post-login and the presence of 'HttpOnly' and 'SameSite=Strict' flags on the cookie.

5. OAuth 2.0 Integration Summary

The application now offers Google OAuth 2.0 sign-in alongside classic credentials. 'oauth/google-login.php' initiates the Authorization Code flow, storing an opaque 'state' token server-side to prevent CSRF. 'oauth/google-callback.php' exchanges the code for tokens, retrieves the OpenID Connect profile, and either links to an existing account or provisions a new record within a database transaction. Session IDs are regenerated after successful login to preserve the hardened authentication posture. Environment variables ('GOOGLE_CLIENT_ID', 'GOOGLE_CLIENT_SECRET', 'GOOGLE_REDIRECT_URI') configure the integration without exposing secrets in source control.

- Register OAuth credentials for the deployment domain in the Google Cloud Console.
- Set the environment variables in the web server configuration or '.env' file (never commit secrets).
- Ensure the callback URL matches 'https://<host>/oauth/google-callback.php' or the configured redirect override.

6. Residual Risks and Deferred Work

- Front-end libraries are still loaded from public CDNs without Subresource Integrity hashes; network-layer tampering could inject malicious scripts. Mitigation: Pin versions with SRI or self-host assets in a future sprint.
- No rate limiting or account lockout is implemented on login endpoints, leaving room for brute-force attempts. Mitigation: Introduce throttling and monitoring middleware.
- Administrative actions beyond login were not in scope for this phase and should undergo the same secure coding review before release.

7. Deployment and Operational Notes

- Set database credentials via 'DB_HOST', 'DB_USER', 'DB_PASSWORD', and 'DB_NAME' environment variables before deploying; the connection bootstrap now reads from the environment and terminates safely on failure.
- Serve the site exclusively over HTTPS so that secure session cookies are honoured and OAuth callbacks succeed.
- Back up the database prior to deploying these schema-affecting changes (new OAuth columns) and verify that cron jobs or background workers, if any, adopt the new connection bootstrap.

8. Validation Summary

- Functional smoke tests for registration, login (local and Google), booking creation, and booking history review passed on the hardened build.
- PHP syntax validation succeeded: 'php -l' was executed against all modified scripts (login, register, booking flow, admin login, shared includes, and OAuth handlers).
- Manual attack attempts (SQL injection payloads, forged CSRF requests, stored XSS strings) now fail gracefully without compromising the system.

9. Conclusion

The remediation effort eliminated seven critical vulnerabilities, modernised authentication storage, and added a secure OAuth 2.0 experience without regressing existing workflows. Remaining items focus on defence-in-depth enhancements such as rate limiting and supply-chain hardening. With the documented configuration steps and residual risk register, the application is ready for submission and further iterative improvements by the development team.